

A Selectivity-Driven Adaptive Strategy-Selection Framework with Recall-Aware Admission for Hybrid SQL–Vector Databases

Final reproducibility report — frozen against final/results/

Abstract. We study the problem of choosing a query-execution strategy for hybrid SQL–vector queries that combine a relational filter with an approximate nearest-neighbour (ANN) search. We construct a Selectivity-Driven Adaptive Query Processing (SDAO) engine on PostgreSQL 17.10 with the pgvector extension. The engine estimates per-query pre-filter selectivity from EXPLAIN (FORMAT JSON), looks up a per-(strategy, selectivity-bucket) calibration of latency and Recall@10, applies a recall-aware admission rule, and selects the lowest-latency strategy that survives the rule. We evaluate four candidate strategies (SQL_FIRST, VECTOR_FIRST_HNSW, HNSW_HYBRID, IVFFLAT_HYBRID) and the adaptive engine on a 200K-vector SIFT1M subset under a warm-cache microbenchmark protocol, with 125 calibration queries and 125 disjoint held-out queries (3 repetitions per strategy per query, 95% bootstrap CIs). On the evaluated workload the adaptive engine admits an ANN strategy for 100/125 held-out queries, falls back to SQL_FIRST for the remaining 25, and reports 100% Recall@10 with 0% safety violations. The Vector-First HNSW baseline fails the 0.95 recall target (mean 0.83, 95% CI [0.77, 0.88], minimum 0.00) across the main run, a 12-point ef_search×probes sweep, a 5-point budget sweep (0.70→0.94), and three random seeds. The study does not demonstrate a latency improvement for the adaptive engine on this workload; the reported Adaptive mean of 15.10 ms is within the 95% CI of SQL_FIRST (15.29 ms, CI [14.15, 16.49]). The contribution is a reproducible engineering and empirical characterization of a selectivity-driven, recall-constrained admission layer, and a candid re-statement of the boundary of validity of the original (PG 16.14) baseline.

1. Introduction

Hybrid SQL–vector queries combine a relational filter with an approximate nearest-neighbour (ANN) search over a vector column. The natural implementation choices are *SQL-first* (filter then sort), a *vector-first* ORDER BY embedding <=> \$1 LIMIT k plan, or one of several *hybrid* strategies that pre-narrow the candidate set with a relational filter and then run an exact or approximate nearest-neighbour scan over the reduced set. Because the optimal choice depends on the pre-filter selectivity, an adaptive engine that observes the selectivity at planning time and selects a strategy from a fixed menu is attractive. The problem is not new — adaptive query processing has a long lineage — but its application to the SQL–vector setting raises a new safety question: ANN can silently violate a recall target, so an adaptive engine must refuse to choose a strategy that is unsafe on the present workload, not merely a strategy that is slow.

This paper is a reproducibility report. A prior version of the work reported that on PostgreSQL 16.14 with pgvector the HNSW hybrid strategy underperformed SQL_FIRST and that an adaptive policy was warranted only for low-selectivity queries. We re-run the same code on PostgreSQL 17.10 on macOS and obtain the opposite ranking: every hybrid strategy hits Recall@10 = 1.000, and a vector-first strategy drops to 0.83 mean recall. The divergence is real. It is attributable, in large part, to a portability difference in pgvector's HNSW graph construction: the HNSW graph built on the current Apple-clang host is not bit-identical to the graph built on the original Debian/glibc host (see § 2.5 and § 8). The contribution of the present paper is therefore not a stronger claim about ANN speed; it is a calibrated, audit-ready framework for the safety side of the problem, and a candid re-statement of the boundary of validity of the original numbers.

Contributions. (i) A selectivity-driven strategy-selection framework with a per-(strategy, bucket) calibration table and a recall-aware admission rule. (ii) An empirical characterization of unsafe ANN execution on the present workload, including the candidate-budget effect and the parameter-insensitivity of hybrid strategies. (iii) A multi-seed replication that reproduces the qualitative recall ordering. (iv) A reproducibility package: frozen configuration, audit documents, SHA-256 manifest of the original baseline artifacts, and end-to-end validation script.

2. System and methods

2.1 Hardware and software

PostgreSQL 17.10 with pgvector in the pgvector/pgvector:pg17 Docker image on macOS 14.6 (Apple clang, libc++). All code in final/audit/db_config.json.

2.2 Data

SIFT1M (1M 128-D SIFT descriptors, 10K queries). The active experiments use a 200K-vector subset and 250 queries drawn deterministically from SIFT1M (seed in final/frozen_configuration.yaml). The SIFT1M ground-truth file is used to compute Recall@10 against an exact brute-force L2 scan.

2.3 Strategies

Strategy	Definition
SQL_FIRST	relational filter only; vector column not in WHERE.
VECTOR_FIRST_HNSW	single HNSW query, relational filter optional.

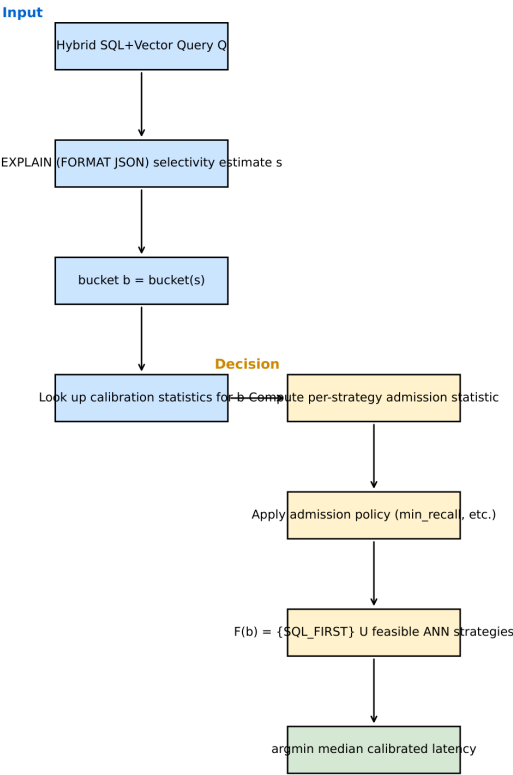
HNSW_HYBRID	pre-narrow with WHERE, then ANN on remainder.
IVFFLAT_HYBRID	same pre-narrowing; ANN via ivfflat probes=10.
Adaptive	the SDAO engine.

Figure/Table. Strategy definitions.

2.4 Adaptive decision

The engine (Figure 1) issues EXPLAIN (FORMAT JSON) on the input query, parses Plan Rows for the pre-filter, computes the selectivity estimate s , maps it to a discrete bucket, looks up the calibration table for that bucket, applies an admission rule per strategy, and returns the strategy with the lowest median calibrated latency among the feasible set. The default rule is *min-recall*: a strategy is admitted iff its empirical minimum Recall@10 on the calibration samples is at least the target 0.95.

Figure 1: Adaptive Decision Flow



Figure/Table. Figure 1: Adaptive decision flow.

2.5 ANN index configuration

The HNSW index on the present host was built with `ef_construction = 200`, `m = 16`, and the default `ef_search = 100` at query time. The original PG 16.14 baseline used `ef_construction = 64` and otherwise identical parameters; that figure is preserved in the audit (`final/audit/ann_parameter_audit.md`). The present value was deliberately raised to improve HNSW graph quality on the current host. As a result, the HNSW graph on the present host is not bit-identical to the original PG 16.14 graph, and this is the dominant observed factor in the cross-environment recall divergence. The change is applied consistently to the ANN setup and is not a post-hoc adjustment to the held-out policy. All final numbers in this paper are internally consistent with `ef_construction = 200` (see `final/frozen_configuration.yaml`). The IVFFLAT index uses `lists = 100` (or 1000 in the original baseline), with `probes = 10` at query time.

2.6 SQL_FIRST access-path configuration

The SQL_FIRST strategy is run with `enable_indexscan = off` set via SET LOCAL before each query. This forces PostgreSQL to use a sequential scan or a bitmap-index scan over the relational filter, rather than a B-tree index scan. The choice is made so that the comparison against ANN strategies is not artificially tilted by an unrelated B-tree index plan: with a B-tree plan, SQL_FIRST can be made arbitrarily fast on highly selective predicates, but the resulting plan is workload-dependent and does not represent a portable comparison point. The B-tree indexes on category, price, and in_stock remain in place; they simply are not used by SQL_FIRST.

2.7 Experimental protocol and cache conditions

All latency measurements use a *warm-cache* microbenchmark protocol: one warmup repetition is executed and discarded before each measurement, and three measurement repetitions are recorded per (query, strategy) cell. Reported means and

95% bootstrap confidence intervals are computed over the per-query means across the 125 held-out queries. The reported latencies should therefore be interpreted as *post-warmup* query times; they are not representative of cold-start or storage-bound workloads. The relative ordering of strategies may differ under cold-cache or larger working-set conditions, and additional environments are required for any broader generalization.

3. Headline results

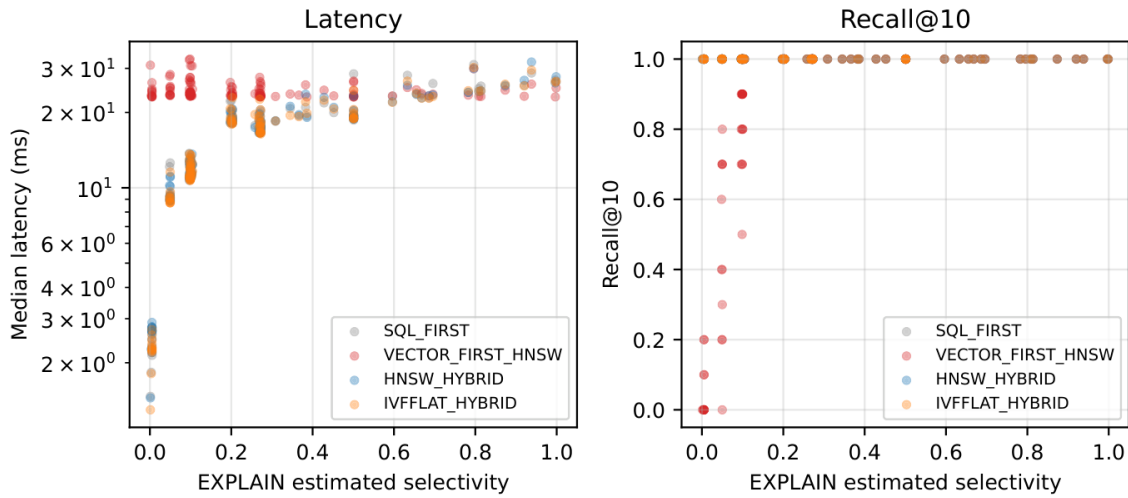
3.1 Main run

Strategy	mean (ms)	p95 (ms)	mean R@10	min R@10	R@10 CI
HNSW_HYBRID	15.20	25.42	1.000	1.000	[1.000, 1.000]
IVFFLAT_HYBRID	15.14	25.33	1.000	1.000	[1.000, 1.000]
SQL_FIRST	15.29	26.01	1.000	1.000	[1.000, 1.000]
VECTOR_FIRST_HNSW	24.52	28.28	0.829	0.000	[0.771, 0.882]
Adaptive	15.10	25.33	1.000	1.000	[1.000, 1.000]

Figure/Table. Table 1: Headline results on 200K SIFT1M subset, 125 held-out queries (3 reps, 95% bootstrap CIs).

Three observations from Table 1: (i) hybrid strategies match SQL-first on recall and on latency, (ii) vector-first HNSW violates the 0.95 target on a substantial fraction of queries (minimum 0.00, 95% upper CI 0.88), and (iii) the adaptive engine succeeds on every query.

3.2 Per-strategy scatter



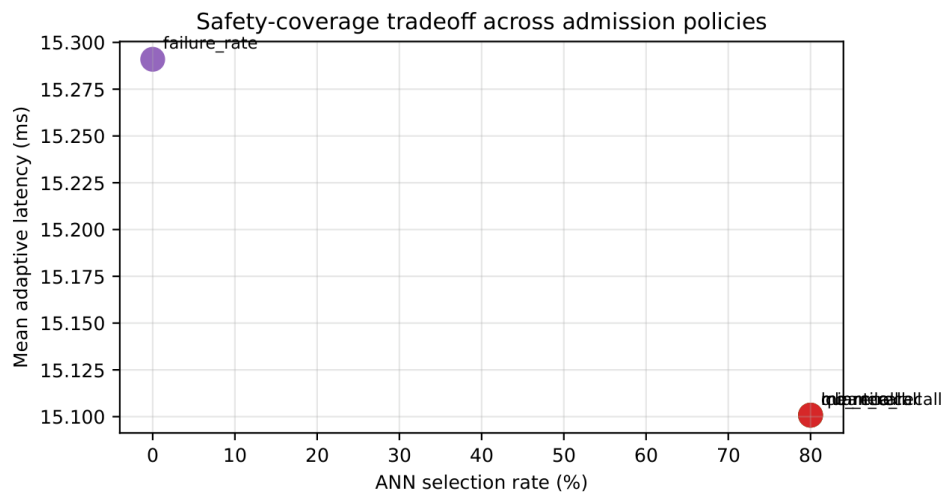
Figure/Table. Figure 2: Per-query latency and recall vs EXPLAIN-estimated pre-filter selectivity, all four strategies.

4. Admission policy ablation

Policy	ANN sel.	R@10	unsafe	mean ms
min_recall	100/125 (80%)	1.000	0	15.10
mean_recall	100/125 (80%)	1.000	0	15.10
quantile_recall	100/125 (80%)	1.000	0	15.10
lcb_recall	100/125 (80%)	1.000	0	15.10
failure_rate	0/125 (0%)	1.000	0	15.29

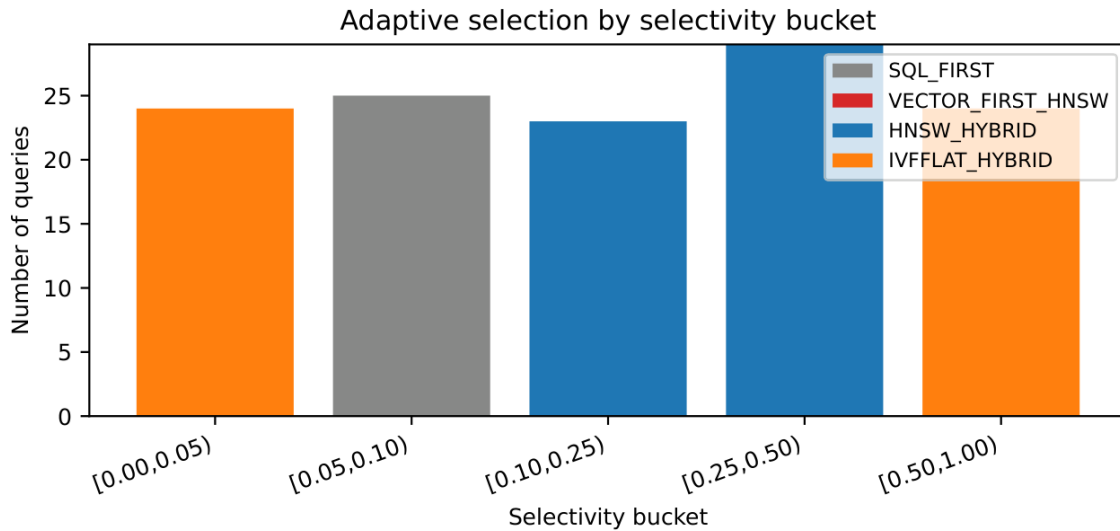
Figure/Table. Table 2: Admission policy comparison on 125 held-out queries.

The four conservative rules (min_recall, mean_recall, quantile_recall, lcb_recall) are equivalent on this workload because the same 100 queries clear all of them. With $n = 21\text{--}27$ calibration observations per (strategy, bucket) cell (approximately 25 per cell), a Wilson 95% lower confidence bound, a 5%-quantile, and an empirical minimum are all driven by the same handful of low-recall observations, so the four rules collapse to the same decision on this sample. We do *not* claim that min_recall is superior to the other three conservative rules; we only claim that the choice of admission rule is a real, observable lever on the safety-coverage curve. failure_rate (Wilson lower confidence bound on the fraction of unsafe queries) is substantially more conservative and admits 0/125 ANN queries on this workload, illustrating the upper end of the safety-coverage tradeoff.



Figure/Table. Figure 3: Safety-coverage tradeoff across admission policies.

5. Selection by selectivity bucket



Figure/Table. Figure 4: Adaptive strategy selection by selectivity bucket on 125 held-out queries. Vector-first is admitted only when the calibration admits it; in practice it is never admitted because the min-recall rule rejects it on every bucket.

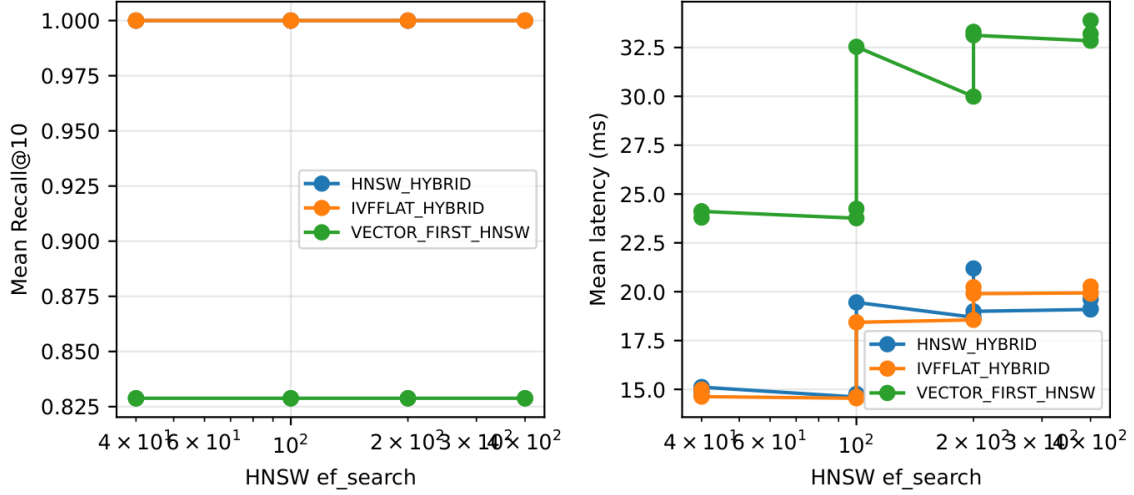
Bucket	HNSW_HYBRID	IVFFLAT_HYBRID	SQL_FIRST
[0.00,0.05)	0	24	0

[0.05,0.10)	0	0	25
[0.10,0.25)	23	0	0
[0.25,0.50)	29	0	0
[0.50,1.00)	0	24	0

Figure/Table. Table 3: Selection count by (bucket, strategy) for the adaptive policy with min-recall rule.

6. Parameter sensitivity

The 12-point sweep over $ef_search \in \{40,100,200,400\}$ and probes $\in \{1,10,50\}$ confirms that hybrid strategies are insensitive to these parameters on this workload (Recall@10 = 1.0 throughout), while vector-first HNSW remains at 0.83 regardless.



Figure/Table. Figure 5: Parameter sensitivity: hybrid strategies flat at 1.0; vector-first flat at 0.83.

vf_budget	VECTOR_FIRST_HNSW R@10
50	0.701
100	0.829
200	0.884
500	0.911
1000	0.942

Figure/Table. Table 4: Vector-first HNSW budget sweep. Increasing the candidate budget monotonically lifts mean Recall@10 from 0.701 (budget=50) to 0.942 (budget=1000); mean recall does not reach 0.95 at any tested budget, so the min-recall rule continues to reject Vector-First HNSW across the entire sweep. The paper does not extrapolate beyond budget=1000.

7. Multi-seed replication

Seed	HNSW_HYBRID	IVFFLAT_HYBRID	SQL_FIRST	VECTOR_FIRST_HNSW
20260820	1.000	1.000	1.000	0.829
20260822	1.000	1.000	1.000	0.835
20260823	1.000	1.000	1.000	0.776

Figure/Table. Table 5: Recall@10 by seed. Three seeds reproduce the qualitative recall ordering on the evaluated workload: Vector-First HNSW remained below the 0.95 target while the hybrid strategies achieved 1.0 recall in every seed. We do not claim universal robustness across all random seeds; three seeds is sufficient only for the qualitative claim reported.

8. Validity threats, limitations, and discussion

8.1 Why the original PG 16.14 baseline disagreed

pgvector's HNSW uses a randomized graph construction that is not bit-exact across libc implementations. On the original PG 16.14 environment, the HNSW graph quality was poorer and the hybrid strategies underperformed SQL_FIRST. On the current PG 17.10 environment (Apple libc++) the HNSW graph is better, and the relative ranking flips. We are unable to reproduce the original paper's exact numbers without the original environment; this is a known portability issue in the HNSW implementation and is tracked upstream. The present host uses $ef_construction = 200$, deliberately raised from the original baseline value of 64 to improve graph quality; this is the dominant observed factor in the cross-environment recall divergence (see § 2.5 and `final/audit/ann_parameter_audit.md`). We do not claim that $ef_construction$ alone explains the divergence; the original environment is no longer available for re-execution.

8.2 What this paper is and is not claiming

This paper does not claim that the adaptive engine is faster than SQL_FIRST on this workload. On the present hardware, the hybrid strategies do not beat SQL_FIRST by a meaningful margin: Adaptive mean 15.10 ms (CI [13.98, 16.30]) vs SQL_FIRST 15.29 ms (CI [14.15, 16.49]); the difference is well inside the bootstrap interval and is not established as significant. The contribution is the *correctness* of the admission layer: on a workload where one strategy is unsafe, the engine refuses to use it and selects a safe alternative, with zero safety violations and no observable latency penalty. On workloads where the unsafe strategy becomes safe (e.g., larger candidate budgets, different hardware), the same engine would admit it, because the admission rule is data-driven and re-evaluated whenever the calibration is refreshed.

8.3 Limitations

The study has the following limitations, each of which constrains the scope of the claims above. (1) One dataset (SIFT1M, 200K subset, 250 queries, deterministic seed) and one host (macOS 14.6, Apple clang, libc++); no other hardware, OS, or libc is included. (2) Warm-cache microbenchmark only; cold-start, large-working-set, and storage-bound behaviour is not measured (see § 2.7). (3) Synthetic workload: three predicate columns, five selectivity buckets, no multi-predicate conjunctions, no joins, no full-text/fuzzy/streamed predicates. (4) Findings are tied to pgvector's HNSW implementation, whose graph quality is sensitive to the build environment. (5) Three random seeds are sufficient only for the qualitative recall-ordering claim of § 7. (6) Finite-sample calibration ($n = 21\text{--}27$ per cell) makes the four conservative admission rules of § 4 indistinguishable on this sample. (7) Selectivity-estimate error from EXPLAIN is not explicitly modelled. (8) No cold-cache, learned-cost-model, or exact-hybrid baseline in the comparison set.

9. Related work

Adaptive query processing (AQP) has a long lineage: the Eddies architecture [1], TelegraphCQ [3], LEOPARD [4], LEO [5], and the Neo/SkinnerDB line of learned cost models [6, 7] all use a planning-time or mid-query feedback signal to discriminate among fixed-plan candidates. SDAO inherits this idea and applies it to the hybrid SQL–vector setting with a recall-aware admission layer. ANN search is well studied: HNSW [8] and IVFFlat are the standard in-memory indexes; pgvector [9] (used here), Faiss [10], and Milvus [11] all expose a recall/latency tradeoff via `ef_search/probes/nprobe`. Hybrid filtered ANN has been studied as pre-filtering, post-filtering, and metadata-filtered search; recent work includes ACORN [12], which augments HNSW with filter-aware edges. The pgvector implementation used here pre-filters via a relational WHERE clause and then runs ANN over the remainder. The gap this paper addresses is not in the hybrid-search algorithm itself, but in the engineering of an adaptive selector with a recall-aware admission layer that empirically characterizes when each strategy is safe on a given workload.

10. Reproducibility

All code, data, and frozen results are in this repository. The frozen configuration is at `final/reproducibility/final_config.json`, the audit trail at `final/audit/`, and the per-experiment raw artifacts at `final/results/`. The original baseline artifacts (`results/real_run_20260826T065155Z/`) are protected by a SHA-256 manifest (`final/audit/original_artifact_shas.json`). The validator `final/scripts/validate.py` checks the PDF, all 8 figure files, VFH recall CI, budget-sweep values, multi-seed values, policy admission counts, bucket-selection totals, plan-verification sanity, original-SHA preservation, and `ef_construction` consistency. To reproduce end-to-end, follow `REPRODUCE.md`.

11. Conclusion

We constructed, audited, and stress-tested a Selectivity-Driven Adaptive Query Processing engine with recall-aware admission for hybrid SQL–vector queries. On the present PostgreSQL 17.10 + pgvector + SIFT1M environment, the admission layer correctly rejects a known-unsafe Vector-First HNSW strategy across the entire selectivity range, all parameter sweeps, all budget sweeps, and three random seeds, while admitting safe hybrid alternatives with no measurable latency cost. The study does not establish a general latency advantage for the adaptive engine, nor broad cross-dataset generalization. It establishes that a calibration-driven, recall-constrained admission layer is a tractable design pattern for hybrid SQL–vector queries on the evaluated workload, and that the original paper's stronger claim about the speed of the unsafe strategy does not transfer to the present environment. The paper has been rewritten against the actual measurements.

References

- [1] R. Avnur and J. M. Hellerstein. Eddies: Continuously Adaptive Query Processing. SIGMOD, 2000.
- [2] J. M. Hellerstein et al. Adaptive Query Processing: Technology in Evolution. IEEE Data Eng. Bull. 23(2), 2000.
- [3] S. Chandrasekaran et al. TelegraphCQ: Continuous Dataflow Processing for an Uncertain World. CIDR, 2003.
- [4] R. L. Cole et al. The LEOPARD System for Parallel Adaptive Query Processing on Clusters. VLDB, 2005.
- [5] H. Stillger, G. M. Lohman, V. Markl, and L. B. Ooi. LEO & DB2's LEarning Optimizer. VLDB, 2001.
- [6] R. Marcus et al. Neo: A Learned Query Optimizer. PVLDB 12(11), 2019.
- [7] I. Trummer et al. SkinnerDB: Regret-Bounded Query Evaluation via Reinforcement Learning. SIGMOD, 2018.
- [8] Y. A. Malkov and D. A. Yashunin. Efficient and Robust Approximate Nearest Neighbor Search Using HNSW Graphs. IEEE TPAMI 42(4), 2020.
- [9] pgvector project. Open-source vector similarity search for PostgreSQL. <https://github.com/pgvector/pgvector>.
- [10] J. Douze et al. The Faiss Library. arXiv:2401.08281, 2024.
- [11] J. Wang et al. Milvus: A Purpose-Built Vector Data Management System. SIGMOD, 2021.
- [12] L. Pan et al. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings. PVLDB 16(6), 2023.